

Verified Diffie–Hellman with Validation Built In

Making invalid states unrepresentable, and what a verified implementation still cannot promise

Contents

Introduction	2
Part I — Foundations	3
1. Types, terms, propositions, and dependent families	3
2. The logical vocabulary used here	3
3. The natural numbers	4
4. Bool, Prop, and Option	4
5. Structures, dependent records, and proof fields	4
6. Diffie–Hellman, and why correctness is monoid algebra	5
7. Safe primes, subgroups, and what validation is for	6
8. Primality certificates	7
9. Specifications as definitions, not programs	7
10. Project mechanics	8
11. Just enough IO to read Main.lean	8
12. The architecture: locked spec, verified core, unverified shell	9
Part II — The task, as given	9
File 1 of 4: DHSafe/Spec.lean (locked)	10
File 2 of 4: DHSafe/Impl.lean (stub)	11
File 3 of 4: the library root, the executable entry, and the placeholder	12
File 4 of 4: Main.lean (placeholder)	12
The prompt given to Claude Code	12
Part III — Reading the specification, piece by piece	13
powModRef and PowModCorrect	14
qFloor	14
GoodGroup	15
DHGroup	15
Secret	16
ValidPublic	16
PublicValue	17
SchemeCorrect	17
KeyCorrect	22
Obligations that live in types, not in theorem statements	22

The three theorems, and the locked section	23
What the prompt asks that the specification cannot express	23
Part IV — The unverified shell: <code>Main.lean</code>	23
<code>bytesToNat</code>	24
<code>freshSecret</code>	25
<code>main</code>	25
Part V — Verification, as actually performed	26
The protocol, in order	26
Step 1: the agent stayed inside its permissions	26
Step 2: mechanical scans instead of reading	26
Step 3: the axiom audit, including the executable	27
Step 4: the program, run twice	27
Step 5: an independent cross-check	28
Step 6: closing the gap in the demonstration	29
Workflow pitfalls encountered	30
What has been established, and what has not	30
Natural next steps	31
References	31

Introduction

What this project is. Diffie–Hellman (DH) key agreement lets two parties, Alice and Bob, who share no secret, agree on one over a public channel. This is the second version of a project that implements DH in Lean 4 with an AI agent (Claude Code) doing the implementation and the proofs. The first version verified only *functional correctness* and left the surrounding discipline to the human reader: nothing forced the caller to validate the other party’s value, nothing checked the group parameters, and one function was never constrained at all. This version moves all of that into the specification.

The goal. To specify a DH scheme precisely enough that an implementation satisfying the specification is *air-tight without being read*: no code path produces a key from an unvalidated peer value or from unvalidated parameters, and the caller cannot bypass either check, because the type system does not allow it.

The central idea. Validation is made a *type*, not a function that someone must remember to call. A value of type `PublicKey` cannot exist unless a proof of its validity exists with it. The shell never decides whether to validate; it receives either nothing or a value that is already valid.

Scope of this document. As in the earlier chapters, the subject is the specification: how to read it, what each clause commits to, and what a green build therefore establishes. The bodies of the functions and the proofs the agent wrote (239 lines) are deliberately not shown: once the specification is fixed and the theorems are proven and axiom-checked, that code is exactly what formal verification exists to spare you from reading. The one exception is `Main.lean`, the small executable that no theorem covers, which is shown and reviewed in full.

What air-tight can and cannot mean. Three properties are enforceable without reading the implementation: functional correctness; no path to a key that skips validation; no API misuse of the kind

a type checker catches. Three things are out of reach of any such specification and remain open however good it is: secrecy of the key (which rests on a hardness assumption), side channels, and the quality of the entropy behind a secret. [Part V](#) states the final accounting.

Part I — Foundations

Every notion below is defined before it is used.

1. Types, terms, propositions, and dependent families

A **type** A may be regarded as an object of a suitably structured category $\mathcal{C}$, and a **term** $t : A$ as a global element $t : \mathbf{1} \rightarrow A$. Prop is a distinguished type whose terms are *propositions*; each proposition P can itself be used as a type, and a **proof** of P is a term of type P . The judgements $P : \text{Prop}$ (“ P is a well-formed statement”) and $t : P$ (“ t proves P ”) live at two levels of one hierarchy.

Function types and currying. $A \rightarrow B$ is the exponential object B^A . Several arguments are written as a chain of arrows, which associate to the right:

$$\text{sugar: Nat} \rightarrow \text{Nat} \rightarrow \text{Nat} \quad \text{bracketed: Nat} \rightarrow (\text{Nat} \rightarrow \text{Nat}).$$

Application is juxtaposition and associates to the left: $f x y$ means $(f x) y$.

Dependent types. A *family* of types indexed by a type B assigns to each $b : B$ a type $F b$; categorically, an object of the slice $\mathcal{C}/B$. The **dependent product** $\prod_{b:B} F b$, written in Lean $(b : B) \rightarrow F b$, is the type of functions whose *result type depends on the argument*; it is the space of sections of the family. The non-dependent $B \rightarrow C$ is the case where F is constant. This project uses the dependent form throughout: a function taking a group G and returning something whose type mentions G .

Higher-order specifications. Since $A \rightarrow B$ is an ordinary type, a definition may take an entire function as a parameter. Every specification here does: it states a property that *any* function of a given type might or might not have, before any particular function has been written. A name such as `public0f` inside a specification is a placeholder for “whichever function eventually fills this role”; only its type is needed to state the property.

2. The logical vocabulary used here

The specification uses only the following.

- $x = y$, $x < y$, $x \leq y$ are propositions about natural numbers; $=$ is also used between values of other types, such as `Bool` and `Option` values ([Section 4](#)).
- $P \wedge Q$ (“and”): a proof is a pair of proofs. It associates to the right, so $A \wedge B \wedge C$ means $A \wedge (B \wedge C)$.
- $P \rightarrow Q$ (“implies”): the function type from proofs of P to proofs of Q .
- $P \leftrightarrow Q$ abbreviates $(P \rightarrow Q) \wedge (Q \rightarrow P)$. A clause stated with $\leftrightarrow$ asserts *both* directions, and a clause missing one direction is satisfied by a useless implementation ([Part III](#) gives examples).
- $\forall(x : A), P x$: a dependent product of proofs. Several variables curry like functions.

Hypotheses weaken a claim. $\forall p, 0 < p \rightarrow Q(p)$ asserts $Q(p)$ only for $p > 0$. Every hypothesis to the left of an arrow is a case the guarantee does not cover.

3. The natural numbers

Nat is the inductive type with constructors `zero` and `succ`: the initial algebra of $X \mapsto \mathbf{1} + X$, a natural numbers object. Its universal property is primitive recursion. Three conventions matter when reading a specification over Nat.

- $x \% p$ is the remainder of x on division by p ; Lean sets $x \% 0 = x$.
- $x - y$ is *truncated*: $x - y = 0$ whenever $y \geq x$. Specifications therefore prefer addition, writing $y + 1 < p$ rather than $y < p - 1$, so that no reader has to ask whether truncation could occur.
- $\wedge$ binds more tightly than $*$ and $\%$, so $g \wedge e \% p$ is $(g^e) \bmod p$.

Natural numbers have arbitrary size: 2^{126} is an ordinary value, computed by an arbitrary-precision library at run time.

Primality. `Nat.Prime p` is Mathlib’s definition of “ p is prime”: $p \geq 2$ and its only divisors are 1 and p (formally, p is irreducible). It is a proposition, not a program: stating it costs nothing, but *proving* it for a 127-bit number is real work (Section 8).

4. Bool, Prop, and Option

`Bool` is *data*: `true` and `false`, on which code can branch. `Prop` is a *claim*. A specification often relates the two:

$$\forall x, f\ x = \text{true} \leftrightarrow P\ x$$

says the computable test f decides P exactly. The same distinction appears in equality: $x = y$ is a proposition, while `x == y` is a `Bool`-valued test that a program can evaluate. For natural numbers the two agree.

`Option  $\beta$` has exactly two shapes of value: `none` (“nothing here”) and `some  $b$` (“something is here, namely b ”). It is the standard type of a computation that may fail to produce a result. `o.isSome` is the `Bool` that is `true` exactly when o is a `some`. Lean functions are total, so a function that rejects bad input cannot throw: it returns `none`, and “never crashes on garbage” is guaranteed by the *return type alone*.

5. Structures, dependent records, and proof fields

This section is the technical heart of the project. It uses a small example of its own; every structure in Part II has the same shape.

Structures are dependent records. A declaration

```
structure InRange (lo hi : Nat) where
  a : Nat
  ok : lo ≤ a ∧ a < hi
```

defines a type whose values bundle named *fields*. A later field may depend on earlier ones: `ok` mentions `a`. Categorically, `InRange lo hi` is the iterated dependent sum $\Sigma (a : \text{Nat}), (lo \leq a \wedge a < hi)$. Fields are read with dot notation: for $r : \text{InRange } lo\ hi$, $r.a$ is the number and $r.ok$ the proof.

The parameters $(lo\ hi : \text{Nat})$ make `InRange` a *family* of types, one for each choice of bounds (Section 1). A structure may equally be parameterised by a value of *another* structure, with its constraints referring to that value’s fields; Part II uses this constantly.

Proof fields make a structure a subset. The field `ok` is a proof. Lean has *proof irrelevance*: any two proofs of the same proposition are equal, definitionally. Hence two values of `InRange lo hi` with the same `a` are equal, and the structure carries no data beyond `a`: it is the subtype $\{a : \text{Nat} \mid lo \leq a < hi\}$, a subset of `Nat`. In the topos reading of Lean’s `Prop` as a subobject classifier Ω , it is the pullback of $\text{true} : \mathbf{1} \rightarrow \Omega$ along the predicate’s classifying map:

$$\begin{array}{ccc} \text{InRange } lo\ hi & \xrightarrow{\quad} & \mathbf{1} \\ \downarrow a & \lrcorner & \downarrow \text{true} \\ \text{Nat} & \xrightarrow{\lambda a. (lo \leq a \wedge a < hi)} & \text{Prop} \end{array}$$

Proofs are erased when the program is compiled, so the invariant costs nothing at run time.

Constructing a value requires every field, proofs included. The *anonymous constructor* $\langle a, h \rangle$ builds a value of `InRange lo hi` from a number `a` and a proof $h : lo \leq a \wedge a < hi$. Since no proof of a false statement exists, **no value of this type violates its invariant**, in any program, anywhere. Compare a function `check : Nat → Bool`: it may be perfectly correct, but nothing forces a caller to invoke it or to heed its answer. A proof field turns “remember to check” into “cannot be skipped”.

Where the guarantee ends. The invariant is airtight only against code that builds the value honestly. Code could still obtain a value by supplying a proof via `sorry` or a newly declared axiom, or by redirecting compiled code with attributes such as `implemented_by`; these are exactly what the mechanical checks of Part V scan for. And a type can only rule out violations of the invariant it *states*: a type saying “in range” says nothing about whether the number was chosen at random.

6. Diffie–Hellman, and why correctness is monoid algebra

Fix a prime p and an element g of $(\mathbb{Z}/p)^\times$. Alice draws a secret a , Bob a secret b . They exchange public values and each combines the other’s value with their own secret:

$$\begin{array}{ccc} \text{Alice} & \text{secret } a & \text{Bob} & \text{secret } b \\ & \xrightarrow{A = g^a \bmod p} & & \\ & \xleftarrow{B = g^b \bmod p} & & \\ K_A = B^a \bmod p & & K_B = A^b \bmod p & \\ & K_A = g^{ab} \bmod p = K_B & & \end{array}$$

An eavesdropper sees p, g, A, B ; computing g^{ab} from them is the *computational Diffie–Hellman problem*, believed hard in well-chosen groups.

Correctness is a fact about monoids. In any monoid M , set $x^0 = 1$ and $x^{n+1} = x^n \cdot x$. Induction gives $(x^a)^b = x^{ab}$; that is, $n \mapsto (-)^n$ is an action of $(\mathbb{N}, \cdot, 1)$ on M . Since $(\mathbb{N}, \cdot)$ is commutative,

$$(x^a)^b = x^{ab} = x^{ba} = (x^b)^a.$$

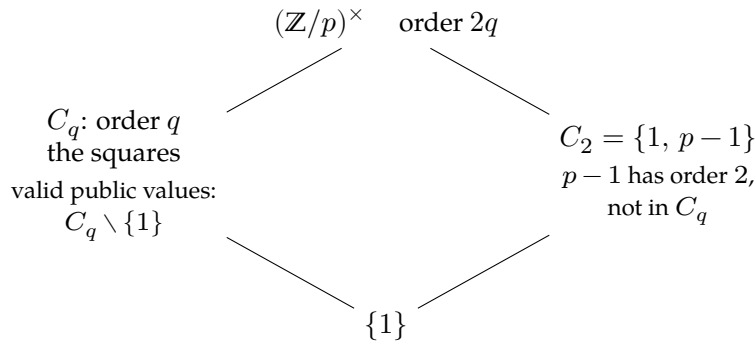
That line is DH agreement, valid in every monoid, with no mention of primes. The quotient $\pi : \mathbb{N} \rightarrow \mathbb{Z}/p$ is a homomorphism of multiplicative monoids, so it commutes with powers:

$$\begin{array}{ccc} \mathbb{N} & \xrightarrow{(-)^e} & \mathbb{N} \\ \pi \downarrow & & \downarrow \pi \\ \mathbb{Z}/p & \xrightarrow{(-)^e} & \mathbb{Z}/p \end{array}$$

so an implementation may reduce modulo p after every multiplication. Correctness is therefore easy and says nothing about security: security lives entirely in the particular group, which is the subject of the next section.

7. Safe primes, subgroups, and what validation is for

For a prime p , the group $(\mathbb{Z}/p)^\times$ is cyclic of order $p - 1$. A prime p is **safe** when $p = 2q + 1$ with q also prime. Then $p - 1 = 2q$, and a cyclic group of order $2q$ has exactly four subgroups, of orders 1, 2, q , and $2q$:



The order- q subgroup C_q is exactly the set of squares, and $y \in C_q \iff y^q \equiv 1 \pmod{p}$.

Why work in C_q . If a party accepts any residue as the other's public value, an attacker can substitute an element of small order. The values 0, 1, and $p - 1$ force the resulting key into a set of at most two elements; in a group with small prime-order subgroups, a substituted element of such an order confines the key to that subgroup and leaks information about the secret. In a safe prime group the only small subgroup is $C_2 = \{1, p - 1\}$, and restricting all public values to $C_q \setminus \{1\}$ excludes it entirely.

Validation. A peer's value y is acceptable exactly when $1 < y < p - 1$ and $y^q \equiv 1 \pmod{p}$ (NIST SP 800-56A "full validation" for such groups; Revision 3 of that standard approves specific safe-prime groups for this purpose). The set of acceptable values has $q - 1$ elements. Secrets are drawn from $[1, q - 1]$, also $q - 1$ values, and if g generates C_q then $a \mapsto g^a \pmod{p}$ is a *bijection* from secrets to acceptable public values. This is why the implementation can be required to prove that its own public value always passes validation.

Why the order q must be large. The best generic attack on the discrete logarithm in a group whose order has largest prime factor ℓ costs about $\sqrt{\ell}$ group operations (Pollard's rho, combined across prime factors by Pohlig–Hellman). In a safe prime group, $\ell = q$, with no smaller prime factor to exploit. The size of q is therefore the quantitative security parameter of the specification, which is why a *floor* on q appears in it.

A cautionary example. The earlier version of this project used the modulus $2^{127} - 1$, a Mersenne prime. Its group order factors as $2 \cdot 3^3 \cdot 7^2 \cdot 19 \cdot 43 \cdot 73 \cdot 127 \cdot 337 \cdot 5419 \cdot 92737 \cdot 649657 \cdot 77158673929$, whose largest prime factor is only 37 bits, so discrete logarithms in it cost a few hundred thousand operations. It was a fine toy for checking correctness and an impossible one for a floor that means anything. The group below is instead a safe prime, with q of 127 bits.

What the toy group does not provide. A 127-bit q gives roughly 2^{63} operations for rho, and the 128-bit modulus is within reach of index-calculus methods that ignore q altogether. The group exists to exercise the proof obligations at realistic structure, not to protect anything. Real deployments use groups of 2048 bits or more, for instance the safe primes of [RFC 7919](#).

8. Primality certificates

Deciding primality of a 127-bit number by trial division is hopeless, but *checking a certificate* is cheap. **Lucas's criterion:** if $n > 1$ and there is an a with

$$a^{n-1} \equiv 1 \pmod{n} \quad \text{and} \quad a^{(n-1)/r} \not\equiv 1 \pmod{n} \quad \text{for every prime } r \mid n-1,$$

then n is prime. (Pratt later observed that applying this recursively gives every prime a short certificate; see [the References](#).) *Sketch.* The order of a divides $n-1$ but divides none of the $(n-1)/r$, so it equals $n-1$. The order also divides $\varphi(n) \leq n-1$, with equality only for primes. Hence $\varphi(n) = n-1$ and n is prime.

The criterion needs the complete prime factorisation of $n-1$. For a safe prime $p = 2q + 1$ this reads $p-1 = 2q$, so proving p prime needs q prime (recursive), and for q one wants $q-1$ to factor into primes small enough to verify by trial division. The group in this project was chosen so that

$$q-1 = 2 \cdot 154373 \cdot 181277 \cdot 203449 \cdot 205391 \cdot 273073 \cdot 321577 \cdot 504901,$$

a product of seven primes of 18 or 19 bits each, giving a certificate of depth two. The prompt in [Part II](#) supplies this factorisation, plus the witnesses a , as a *hint*. The hint is not trusted: if any claim in it were wrong, the agent's Lean proof would fail to build.

9. Specifications as definitions, not programs

A specification may describe a result by an expression that is transparent but computationally useless. The specification in [Part II](#) defines `powModRef  $g \in p$` as $g^e \bmod p$ computed literally, by forming g^e first. With an exponent of around 2^{126} that number has about 10^{37} decimal digits, vastly more than all the digital storage on Earth could hold, so it can never be *run*; as a *definition* it is perfect. An efficient implementation is required to agree with it everywhere, a **refinement** theorem.

What refinement cannot express. Agreement of outputs says nothing about running time. An implementation that called the reference would satisfy the same theorem and never finish. Running the program at realistic sizes is therefore a check on a property the specification cannot state, and `powModRef` appears in this project only in statements and proofs, never in executable code.

Obligations can live in types. A function whose *output type* is a proof-carrying structure carries a proof obligation that appears in no theorem statement. Reading a specification therefore includes reading the types of the functions it constrains ([Part III](#) returns to this).

10. Project mechanics

Modules and imports. A file's module name is its path relative to the project root with `/` replaced by `·`: the file `DHSafe/Spec.lean` is the module `DHSafe.Spec`, and `import DHSafe.Spec` brings its declarations into scope. `import Mathlib` brings in the whole `Mathlib` library, which supplies `Nat.Prime`. A library is registered with `lake`, Lean's build tool, by a `[[lean_lib]]` entry in `lakefile.toml`, and an executable by `[[lean_exe]]`.

Namespaces. `namespace Impl ... end Impl` places the enclosed declarations under the prefix `Impl`, so `powMod` defined there is `Impl.powMod` elsewhere.

Definitions, theorems, placeholders. `def f : T := t` defines `f`; theorem `name : P := proof` has the same shape with a proposition as its type. `sorry` is a placeholder accepted with a warning. `/- - ... -/` is a documentation comment, and `--` a line comment.

The locked-theorem pattern. The stub ends with theorems whose type is identical to an earlier theorem's and whose proof just refers to it. This mechanically prevents a proof of a *different*, weaker statement from being substituted. It does not prevent the *definitions* from being edited, since a theorem's type refers to whatever the definitions currently say.

Locking for real. That gap is closed outside Lean. `Spec.lean` is written by hand, built, committed to a git repository, and made immutable with `sudo chattr +i`; the build configuration (`lakefile.toml`, `lean-toolchain`, `lake-manifest.json`) is locked too, so the agent cannot change compiler options. The agent has no administrator rights. Afterwards an empty `git diff` proves the specification is byte-for-byte the committed one.

11. Just enough IO to read `Main.lean`

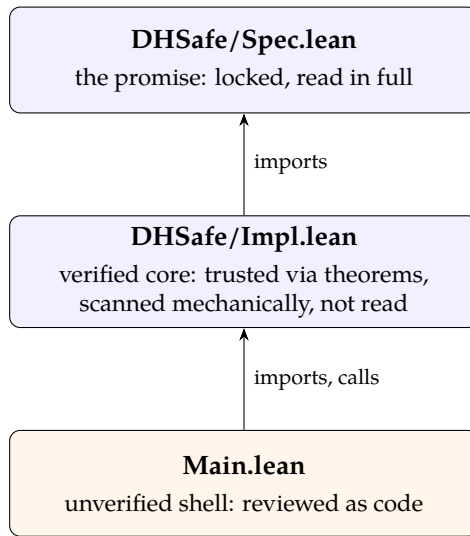
A program that touches the outside world has a type `IO  $\alpha$` : a computation that, when run, may perform effects and then yields a value of type `$\alpha$` . It can be modelled as a pure function `RealWorld  $\rightarrow$   $\alpha \times$  RealWorld` threading a token for the state of the world from one action to the next, which is how Lean stays pure. `Unit` is the type with one value, and `UInt32` is a 32-bit unsigned integer; a program of type `IO UInt32` returns a process exit code, by convention 0 for success.

A `do` block lists actions in order. Inside it:

- `let x := e` names the value of a pure expression `e`;
- `let x ← e` runs the action `e` and names its result;
- `let some x ← e | alt` runs the action `e`; if the result is `some x` it continues with `x` bound, and if it is `none` it runs `alt` instead (in `Main.lean`: print an error and return early);
- `let some x := e | alt` does the same for a *pure* expression `e` of an `Option` type;
- `if c then u else v` chooses between two values according to a `Bool c`;
- `for y in list do ...` runs a block once per element of a list;
- `return v` ends the block with the value `v`;
- `IO.println s` prints a line to standard output, `IO.eprintln s` to standard error.

Two further pieces of notation: `fun x y => e` is an anonymous function (a λ -term), and `s!"... {e} ..."` is *string interpolation*, which builds a string by inserting the textual form of each braced expression.

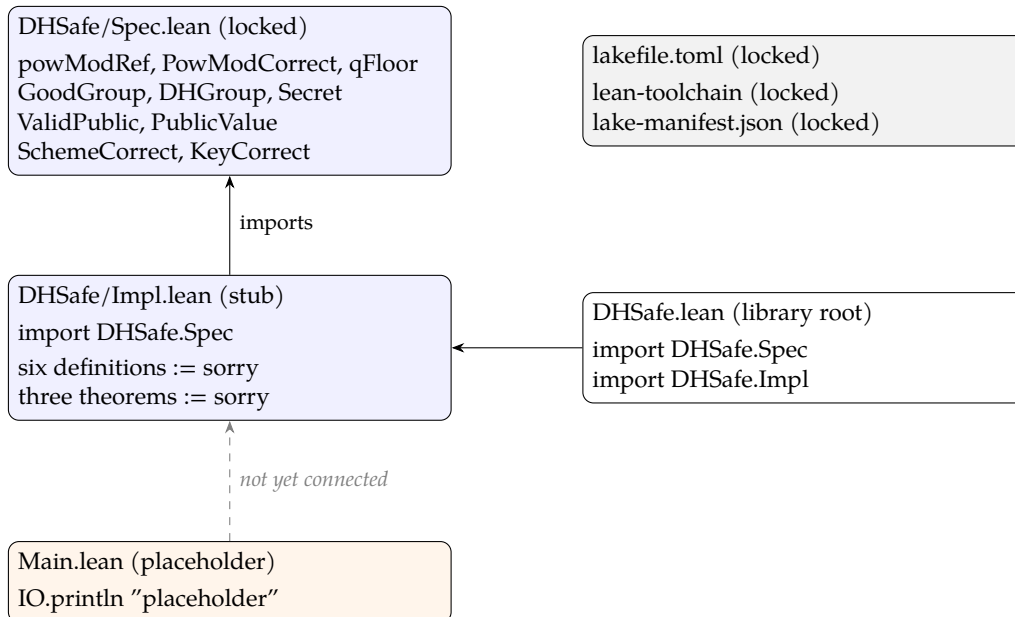
12. The architecture: locked spec, verified core, unverified shell



Every guarantee lives in the top two boxes. `Main.lean` carries no proof, by design: no proposition says “prints the right things in the right order”. What changes in this project is how *little* the bottom box has to do. Because validation and parameter checking are carried by types ([Section 5](#)), the shell cannot skip them. Its remaining duties are to obtain entropy, to move numbers between parties, and to pair each party’s secret with the *other* party’s value. The last is not something types can check, since both parties’ values have the same type.

Part II — The task, as given

Four files existed before the agent started, plus the prompt it received.



Only one real dependency exists at the start: `Impl.lean` imports `Spec.lean`, a file already finished that never changes. `Main.lean` is a placeholder, disconnected from the others; the import it needs is something the agent must add. The repository’s history has two commits made before the agent started: `spec + stubs`, and `disable Mathlib style linters` (the Mathlib project template turns on style warnings that would make a “zero warnings” requirement unsatisfiable on a file the agent may not touch).

File 1 of 4: `DHSafe/Spec.lean` (locked)

```
import Mathlib

/-- Reference modular exponentiation: mathematically transparent,
    computationally infeasible for large exponents. Specification only: never evaluate. -/
def powModRef (g e p : Nat) : Nat := g ^ e % p

def PowModCorrect (powMod : Nat → Nat → Nat → Nat) : Prop :=
  ∀ (g e p : Nat), 0 < p → powMod g e p = powModRef g e p

/-- Size floor on the subgroup order. TOY VALUE: with  $p = 2q + 1$  it forces  $p > 2^{127}$ ,
    far below the 2048-bit / 224-bit minimum for real use. -/
def qFloor : Nat := 2 ^ 126

/-- Acceptable domain parameters: a safe prime  $p = 2q + 1$ , and a generator  $g$  of the
    order- $q$  subgroup ( $g \neq 1$  and  $g^q \equiv 1$  with  $q$  prime force order exactly  $q$ ). -/
def GoodGroup (p g q : Nat) : Prop :=
  Nat.Prime p ∧ Nat.Prime q ∧ p = 2 * q + 1 ∧ qFloor < q ∧
  1 < g ∧ g < p ∧ powModRef g q p = 1

structure DHGroup where
  p : Nat
  g : Nat
  q : Nat
  good : GoodGroup p g q

structure Secret (G : DHGroup) where
  a : Nat
  inRange : 1 ≤ a ∧ a < G.q

/-- Full public-value validation (NIST SP 800-56A): in range and in the order- $q$  subgroup. -/
def ValidPublic (G : DHGroup) (y : Nat) : Prop :=
  1 < y ∧ y + 1 < G.p ∧ powModRef y G.q G.p = 1

structure PublicValue (G : DHGroup) where
  y : Nat
  valid : ValidPublic G y

def SchemeCorrect
```

```

(mkSecret      : (G : DHGroup) → Nat → Option (Secret G))
(publicOf      : (G : DHGroup) → Secret G → PublicValue G)
(parsePublic   : (G : DHGroup) → Nat → Option (PublicValue G))
(shared        : (G : DHGroup) → Secret G → PublicValue G → Nat) : Prop :=
(∀ (G : DHGroup) (a : Nat) (s : Secret G), mkSecret G a = some s → s.a = a) ∧
(∀ (G : DHGroup) (a : Nat), (mkSecret G a).isSome = true ↔ (1 ≤ a ∧ a < G.q)) ∧
(∀ (G : DHGroup) (s : Secret G), (publicOf G s).y = powModRef G.g s.a G.p) ∧
(∀ (G : DHGroup) (y : Nat) (P : PublicValue G), parsePublic G y = some P → P.y = y) ∧
(∀ (G : DHGroup) (y : Nat), (parsePublic G y).isSome = true ↔ ValidPublic G y) ∧
(∀ (G : DHGroup) (s : Secret G) (P : PublicValue G), shared G s P = powModRef P.y s.a G.p)

/-- The headline guarantee: both parties derive g^(ab) mod p. -/
def KeyCorrect
  (publicOf : (G : DHGroup) → Secret G → PublicValue G)
  (shared   : (G : DHGroup) → Secret G → PublicValue G → Nat) : Prop :=
  ∀ (G : DHGroup) (sA sB : Secret G),
    shared G sA (publicOf G sB) = powModRef G.g (sA.a * sB.a) G.p

```

File 2 of 4: DHSafe/Impl.lean (stub)

(One line below is broken in two to fit the page; in the file it is a single line, which Lean reads identically.)

```

import DHSafe.Spec

namespace Impl

def powMod (g e p : Nat) : Nat := sorry
def defaultGroup : DHGroup := sorry
def mkSecret (G : DHGroup) (a : Nat) : Option (Secret G) := sorry
def publicOf (G : DHGroup) (s : Secret G) : PublicValue G := sorry
def parsePublic (G : DHGroup) (y : Nat) : Option (PublicValue G) := sorry
def shared (G : DHGroup) (s : Secret G) (P : PublicValue G) : Nat := sorry

theorem powMod_correct_impl : PowModCorrect powMod := sorry
theorem scheme_correct_impl : SchemeCorrect mkSecret publicOf parsePublic shared := sorry
theorem key_correct_impl : KeyCorrect publicOf shared := sorry

-- locked section: guards against weakening the theorems above
theorem powMod_correct : PowModCorrect powMod := powMod_correct_impl
theorem scheme_correct : SchemeCorrect mkSecret publicOf parsePublic shared :=
  scheme_correct_impl
theorem key_correct : KeyCorrect publicOf shared := key_correct_impl

end Impl

```

File 3 of 4: the library root, the executable entry, and the placeholder

DHSafe.lean:

```
import DHSafe.Spec
import DHSafe.Impl
```

appended to lakefile.toml (the Mathlib template had already declared the library and the dependency):

```
[[lean_exe]]
name = "dhsafe"
root = "Main"
```

File 4 of 4: Main.lean (placeholder)

```
def main : IO Unit := IO.println "placeholder"
```

The prompt given to Claude Code

This is a Lean 4 + Mathlib project at /home/matan/code/lean/DHSafe (its own git repository). These files are immutable: DHSafe/Spec.lean, lakefile.toml, lean-toolchain, lake-manifest.json. Do not modify them or work around their immutability. If you need compiler options such as maxHeartbeats or maxRecDepth, set them with set_option inside DHSafe/Impl.lean. If you believe the spec needs changing, stop and tell me why. Do not run git commit, reset, checkout, or any other command that changes git state. DHSafe/Impl.lean is a stub for you to complete. Main.lean is a placeholder to replace entirely.

Part 1 -- DHSafe/Impl.lean. Replace every sorry and prove the three *_impl theorems.

- powMod: square-and-multiply, reducing mod p at every step. Define it so the Lean kernel can EVALUATE it (e.g. structural recursion on a fuel parameter); well-founded recursion is irreducible and will block decide/rfl, and you will need kernel evaluation below.
- defaultGroup: build a DHGroup, proving GoodGroup in Lean. Hint from me, to be verified by you and used only if every claim checks:
p = 207386738379149228107986191036893352879
q = 103693369189574614053993095518446676439
g = 4
q - 1 = 2 * 154373 * 181277 * 203449 * 205391 * 273073 * 321577 * 504901
Lucas witness for q is 13; for p (where p - 1 = 2q) it is 11.
Prove Nat.Prime q and Nat.Prime p via a Lucas-style criterion from Mathlib (find the exact name with #check / exact?; do not trust my memory of it), using norm_num for the small factors. Prove the g^q condition by relating powModRef to your powMod via powMod_correct_impl and evaluating in the kernel. If any hint value fails, stop and report; do not substitute silently.
- mkSecret, parsePublic: computable, matching the spec exactly. parsePublic must use powMod (never powModRef) for the subgroup check $y^q \bmod p = 1$.
- publicOf: returns $g^a \bmod p$, and must PROVE the result satisfies ValidPublic

```

(nontrivial: g has order exactly q, so  $g^a$  is neither 1 nor  $p-1$  for  $1 \leq a < q$ ).
- shared: returns powMod P.y s.a G.p.
- Prove powMod_correct_impl, scheme_correct_impl, key_correct_impl.
- powModRef may appear ONLY in statements and proofs, never in executable code.
- Forbidden anywhere in the project: sorry, admit, native_decide, new axioms,
  @[implemented_by], @[extern], unsafe, dbgTrace/dbg_trace.

```

Part 2 -- `Main.lean`.

```

- import DHSafe.Impl. Let G := Impl.defaultGroup.
- Never construct DHGroup, Secret or PublicValue directly; obtain every such
  value only through Impl functions.
- Secrets: read 32 bytes from the OS with IO.getRandomBytes (check the exact
  name/signature), convert to a Nat n, use a := n % (G.q - 1) + 1, and call
  Impl.mkSecret. On none, print an error and exit with a nonzero code.
- Alice and Bob each compute their PublicValue with Impl.publicOf. Simulate
  transmission by passing only the plain Nat (.y); the receiver MUST call
  Impl.parsePublic on it and abort on none. Then each computes the key with
  Impl.shared (own secret, peer's parsed value). Print both keys and whether
  they are equal.
- Print p, q, g, both secrets (label them "demo only"), both public values.
- Print a rejection table: for y in [0, 1, G.p - 1, G.p, G.p + 1, 2, 3, G.g]
  print y and whether Impl.parsePublic accepts it.
- Never evaluate powModRef.

```

Run lake build after every change and iterate autonomously until the WHOLE project builds with zero errors and zero warnings. Use `#check` and `exact?` to confirm lemma names rather than guessing. If a proof route looks infeasible, stop and report; never weaken anything to make it pass.

When done:

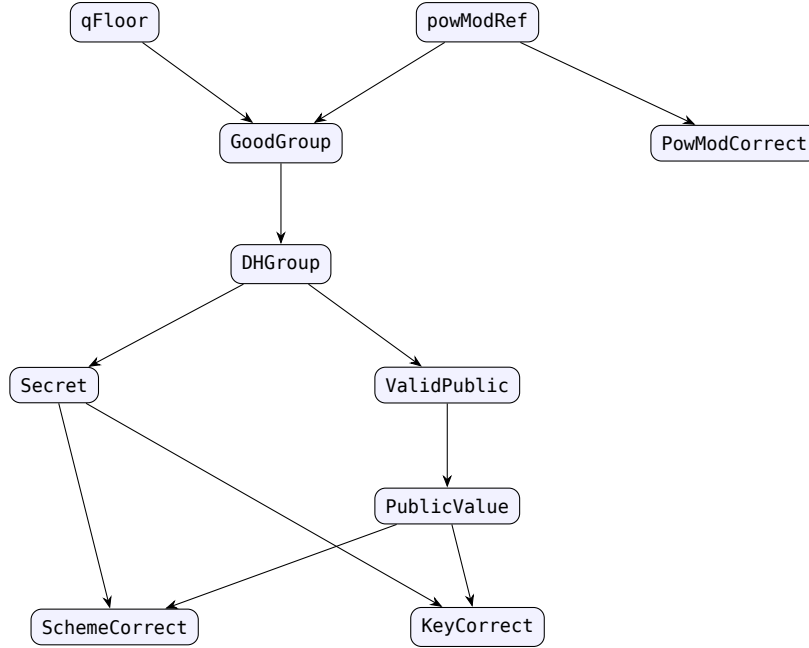
```

- Run #print axioms on Impl.powMod_correct_impl, Impl.scheme_correct_impl,
  Impl.key_correct_impl and on main. Only propext, Classical.choice, Quot.sound
  are acceptable.
- Run lake exe dhsafe and show the output.
- Show me the final DHSafe/Impl.lean and Main.lean.

```

Part III — Reading the specification, piece by piece

The definitions depend on one another as follows. Each node's meaning is built from the nodes above it; `powModRef` is also used inside `ValidPublic`, `SchemeCorrect`, `KeyCorrect`, and `PowModCorrect`, and those edges are omitted.



powModRef and PowModCorrect

```

def powModRef (g e p : Nat) : Nat := g ^ e % p

def PowModCorrect (powMod : Nat → Nat → Nat → Nat) : Prop :=
  ∀ (g e p : Nat), 0 < p → powMod g e p = powModRef g e p

```

Syntax. `powModRef` has type $\text{Nat} \rightarrow \text{Nat} \rightarrow \text{Nat} \rightarrow \text{Nat}$; by [Section 3](#)’s precedence rule its body is $(g^e) \bmod p$. `PowModCorrect` is higher-order ([Section 1](#)): it takes a function of the same type and returns a proposition, with the single hypothesis $0 < p$.

Intuition. `powModRef` is the textbook meaning of modular exponentiation, written as literally as possible. `PowModCorrect` says an efficient `powMod` agrees with it on every input with nonzero modulus: the refinement theorem of [Section 9](#).

Connections. `powModRef` is the common currency of the whole specification: every later clause is stated in terms of it. It is never evaluated; it appears only inside propositions. `PowModCorrect` stands slightly apart: it is used by the implementation as a *bridge*, letting a proof about the infeasible reference be transferred to the efficient function.

qFloor

```

def qFloor : Nat := 2 ^ 126

```

Syntax. A constant natural number.

Intuition. The minimum acceptable size of the subgroup order q . It is the one place where *quantitative* security enters the specification.

Connections. Used only by `GoodGroup`. **It is the line a reader must scrutinise hardest.** Without a floor, the specification would admit $p = 23$, $q = 11$, $g = 2$: every clause satisfied by a group

anyone can break by hand. Like the vacuity problem of earlier chapters, that failure would be invisible in every theorem.

The comment above the definition deserves an equally careful reading, because it invites a wrong fix. “2048-bit / 224-bit” refers to NIST’s minimum sizes for 112-bit security in finite-field groups: a modulus p of 2048 bits with a subgroup order q of at least 224 bits (NIST SP 800-57; see [the References](#)). In general groups these are independent parameters, with q much smaller than p . In *this* specification they are not: clause 3 of `GoodGroup` fixes $p = 2q + 1$. Raising `qFloor` to 2^{223} to “match the comment” would therefore permit a 225-bit modulus, far below 2048 bits and easy prey for index calculus, because the floor constrains only q . For safe-prime groups the floor that forces a 2048-bit modulus is 2^{2046} . A reader who changed the constant by following its own documentation would weaken the specification while believing they had strengthened it, and no build would ever say so.

GoodGroup

```
def GoodGroup (p g q : Nat) : Prop :=
  Nat.Prime p ∧ Nat.Prime q ∧ p = 2 * q + 1 ∧ qFloor < q ∧
  1 < g ∧ g < p ∧ powModRef g q p = 1
```

Syntax. A predicate of three natural numbers, $\text{Nat} \rightarrow \text{Nat} \rightarrow \text{Nat} \rightarrow \text{Prop}$, defined as a right-nested conjunction of seven clauses ([Section 2](#)).

Intuition. Clause by clause:

1. `Nat.Prime p`: the modulus is prime, so $(\mathbb{Z}/p)^\times$ is a cyclic group of order $p - 1$ ([Section 7](#)).
2. `Nat.Prime q`: the subgroup order is prime. This is what makes “order dividing q and not 1” force “order exactly q ”, and it is used inside the proof that a party’s own public value is valid.
3. `p = 2 * q + 1`: p is a *safe* prime. $p - 1 = 2q$ has no small odd factor, so the only small subgroup is $\{1, p - 1\}$ and Pohlig–Hellman has nothing to exploit.
4. `qFloor < q`: the size floor. Together with clause 3 it gives $p > 2^{127}$, so there is no separate floor on p .
5. `1 < g`: excludes $g = 0$ and $g = 1$, whose powers are constant.
6. `g < p`: g is a canonical residue.
7. `powModRef g q p = 1`: $g^q \equiv 1$. With clause 5 and the primality of q , this says g has order *exactly* q , so it generates the subgroup C_q . Note it does not say g generates the whole group, which is impossible for an element of order q when the group has order $2q$.

Connections. The predicate is *easy to state and hard to decide*: clauses 1 and 2 are propositions about 127-bit numbers. It is never decided; it is proved once, for one concrete triple, by the implementation, using the certificate machinery of [Section 8](#). The next definition turns this proposition into something the rest of the project cannot avoid.

DHGroup

```
structure DHGroup where
  p : Nat
  g : Nat
  q : Nat
```

```
good : GoodGroup p g q
```

Syntax. A structure (Section 5) with four fields; the last, `good`, is a proof of `GoodGroup p g q`, depending on the three before it. It is the subtype of triples satisfying `GoodGroup`, a subset of Nat^3 .

Intuition. A domain-parameter set that has been certified good. There is no way to hold a `DHGroup` without having proved it good.

Connections. Every later definition takes a `DHGroup` as its first argument, so *no operation of the scheme can be applied to unvetted parameters*. This also means that the specification never says which group is used: any certified-good group would satisfy it, and the reader learns the actual group only by looking at what the implementation supplies (`defaultGroup`) or at what `Main.lean` prints.

Secret

```
structure Secret (G : DHGroup) where
  a : Nat
  inRange : 1 ≤ a ∧ a < G.q
```

Syntax. A structure *parameterised* by a group `G : DHGroup` (Section 5): for each `G`, a type `Secret G`. `G.q` is the field `q` of `G`.

Intuition. A private exponent in $[1, q - 1]$. The bounds are exactly right: $a = 0$ would give public value 1, and $a = q$ also gives $g^q \equiv 1$.

Connections. The type is indexed by `G`, so a secret for one group cannot be used with another. It records only the *range*; whether the number was chosen at random is not something a type can say. That obligation belongs to the shell, and `Main.lean` (Part IV) is where it is discharged, by reading from the operating system’s entropy source.

ValidPublic

```
def ValidPublic (G : DHGroup) (y : Nat) : Prop :=
  1 < y ∧ y + 1 < G.p ∧ powModRef y G.q G.p = 1
```

Syntax. A predicate on a group and a number, $\text{DHGroup} \rightarrow \text{Nat} \rightarrow \text{Prop}$. Note $y + 1 < G.p$ rather than $y < G.p - 1$, avoiding truncated subtraction (Section 3).

Intuition. The acceptable public values: in $(1, p - 1)$ and in the order- q subgroup (Section 7). Each conjunct rules something out:

- $1 < y$ excludes 0 and 1.
- $y + 1 < G.p$ makes y a canonical residue below $p - 1$. Without it, a value such as $y = p + 1$ passes the subgroup test (it is congruent to 1) while not being a canonical residue at all. It also excludes $y = p - 1$, which the third conjunct would exclude anyway, since $(p - 1)^q \equiv -1$ for odd q .
- $\text{powModRef } y \text{ G.q G.p} = 1$ is subgroup membership, $y^q \equiv 1$.

Connections. This is the predicate that `PublicValue` internalises and that `parsePublic` is required to decide. It is stated with `powModRef`, so it cannot be *run* on 127-bit exponents; the implementation’s

parsePublic must decide it with the efficient powMod, and the specification connects the two (clause 5 of SchemeCorrect).

PublicValue

```
structure PublicValue (G : DHGroup) where
  y : Nat
  valid : ValidPublic G y
```

Syntax and intuition. A number together with a proof that it passes validation: a subtype of exactly the shape of [Section 5](#)’s pullback square, with ValidPublic G as the predicate. A peer’s value that has been checked.

Connections. This is where the central idea of the project becomes concrete. Among the scheme’s operations, the only ones that return a PublicValue are publicOf and parsePublic (both constrained below); any other code would have to supply a proof of validity itself ([Section 5](#)). And shared accepts a PublicValue, not a bare number. Consequently **no key can be computed from an unvalidated value, and the shell has no way to ask for one**. Because the field y is just a natural number, the wire format is unchanged: Alice sends .y and Bob re-validates on receipt, as Main.lean does.

SchemeCorrect

```
def SchemeCorrect
  (mkSecret      : (G : DHGroup) → Nat → Option (Secret G))
  (publicOf      : (G : DHGroup) → Secret G → PublicValue G)
  (parsePublic   : (G : DHGroup) → Nat → Option (PublicValue G))
  (shared        : (G : DHGroup) → Secret G → PublicValue G → Nat) : Prop :=
  (∀ (G : DHGroup) (a : Nat) (s : Secret G), mkSecret G a = some s → s.a = a) ∧
  (∀ (G : DHGroup) (a : Nat), (mkSecret G a).isSome = true ↔ (1 ≤ a ∧ a < G.q)) ∧
  (∀ (G : DHGroup) (s : Secret G), (publicOf G s).y = powModRef G.g s.a G.p) ∧
  (∀ (G : DHGroup) (y : Nat) (P : PublicValue G), parsePublic G y = some P → P.y = y) ∧
  (∀ (G : DHGroup) (y : Nat), (parsePublic G y).isSome = true ↔ ValidPublic G y) ∧
  (∀ (G : DHGroup) (s : Secret G) (P : PublicValue G), shared G s P = powModRef P.y s.a G.p)
```

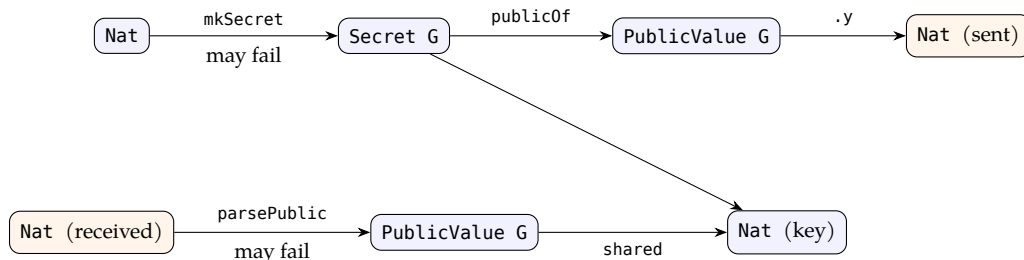
Overall syntax. A higher-order definition over *four* functions, each with a *dependent* type ([Section 1](#)): the result type mentions the group argument. For instance mkSecret : (G : DHGroup) → Nat → Option (Secret G) takes a group and a number and returns an optional secret *for that group*. The body is a six-fold conjunction. Every clause begins by quantifying over all groups G and over the other inputs it needs, so each is a statement about the functions on *every* input, not on examples. The clauses come in pairs: two for mkSecret, one for publicOf, two for parsePublic, and one for shared. Each is read below in three parts.

Overall intuition. The four functions are the whole interface of the scheme: create a secret, compute the value to send, check a received value, compute the key. SchemeCorrect says what each must do, precisely enough that together they determine all four completely (as shown at the end of this section).

The four parameters

`mkSecret`, `publicOf`, `parsePublic`, and `shared` are not definitions. They are *parameters* of `SchemeCorrect`: placeholders for whichever four functions are eventually supplied (Section 1). By itself, `SchemeCorrect` is a predicate on quadruples of functions, a map from the product of four function types to `Prop`. It becomes a single proposition only when four actual functions are plugged in, which happens in the stub’s theorem `scheme_correct_impl : SchemeCorrect mkSecret publicOf parsePublic shared`, where the names refer to the implementation’s functions.

Their types alone already say a great deal. Here is how they move values between types, from one party’s point of view:



Blue boxes are types that carry an invariant; orange boxes are plain numbers that cross the boundary of the program. The only ways *into* the blue region are the two arrows marked “may fail”, and both are among the four parameters. `shared` takes two inputs, one secret and one peer value, which is why two arrows meet at the key.

Parameter 1: `mkSecret`

Syntax. `mkSecret : (G : DHGroup) → Nat → Option (Secret G)`. Bracketed: `(G : DHGroup) → (Nat → Option (Secret G))`. It is a *dependent* function: the result type mentions the first argument, so `mkSecret G` and `mkSecret G'` have different result types when $G \neq G'$. Applying it to a group yields an ordinary function from natural numbers to optional secrets for that group. The result is an `Option` (Section 4) because some numbers must be refused.

Intuition. Turns a raw number, for instance one made from random bytes, into a secret for the group, or refuses it.

Connections. Constrained by clauses 1 and 2. A value of `Secret G` could also be built directly with the anonymous constructor, but that requires writing a proof of the range condition by hand (Section 5); `mkSecret` decides the condition so that callers never need to. In `Main.lean`, `freshSecret` calls it, and its `Option` result is what forces the program to handle a failure branch (Part IV).

Parameter 2: `publicOf`

Syntax. `publicOf : (G : DHGroup) → Secret G → PublicValue G`. Bracketed: `(G : DHGroup) → (Secret G → PublicValue G)`. There is no `Option`: the function is total and always produces a value. Its input is already a secret *for this group*, so the group of the secret and the group of the result agree by construction.

Intuition. Computes the value a party sends to the other: its public key.

Connections. Constrained by clause 3, which fixes the number. Its *return type* carries an obligation that no clause states: every result must come with a proof of `ValidPublic`, so the implementation had to prove that $g^a \bmod p$ is always valid ([Obligations that live in types](#)). In `Main.lean`, the `.y` field of its result is what goes on the wire. It also appears in `KeyCorrect`.

Parameter 3: `parsePublic`

Syntax. `parsePublic : (G : DHGroup) → Nat → Option (PublicValue G)`. The same shape as `mkSecret`, with public values in place of secrets.

Intuition. The gate through which untrusted input enters: a number received from the other party becomes a validated public value, or is rejected.

Connections. Constrained by clauses 4 and 5. Its return type guarantees the safety half of validation on its own: anything it accepts carries a validity proof. Short of writing such a proof by hand, it is the only way to turn a received number into a `PublicValue`, and hence the only way to reach shared from the network. In `Main.lean`, each receiver calls it before doing anything else with the peer’s number.

Parameter 4: `shared`

Syntax. `shared : (G : DHGroup) → Secret G → PublicValue G → Nat`. Bracketed: `(G : DHGroup) → (Secret G → (PublicValue G → Nat))`. Three arguments: a group, one’s own secret, and a peer value. No `Option`: both inputs are already valid, so there is nothing to refuse. The result is a plain natural number, since nothing downstream in this specification needs the key to carry an invariant.

Intuition. Key derivation: combine one’s own secret with the other party’s validated public value.

Connections. Constrained by clause 6, and the subject of `KeyCorrect`. Because its second and third arguments have different types, passing them in the wrong order is a type error. What the types cannot catch is passing one’s *own* public value instead of the peer’s, since both are `PublicValue G`; that is why the pairing in `Main.lean` is reviewed by hand ([Part IV](#)).

Clause 1: `mkSecret` returns the number it was given

```
∀ (G : DHGroup) (a : Nat) (s : Secret G), mkSecret G a = some s → s.a = a
```

Syntax. Three quantified variables: a group, a natural number, and a secret for that group. The hypothesis `mkSecret G a = some s` is an equation between two values of type `Option (Secret G)`: “the call returned some `s`”. The conclusion `s.a = a` is an equation between natural numbers, with `s.a` the number field of the structure ([Section 5](#)).

Intuition. If `mkSecret` succeeds on `a`, the secret it returns contains `a` itself, not some other number.

Connections. Without this clause, the following would satisfy every other clause, and hand every caller the secret 1:

```
mkSecret G a := if 1 ≤ a ∧ a < G.q then some (1, _) else none
```

(the `_` stands for the required proof, which an implementation could supply). A party’s public value would then be g , and its key $B^1 = B$ would be the *other party’s public value*, which travels

in the clear. The clause says nothing about *when* `mkSecret` succeeds: a function that always returns `none` satisfies it vacuously. That is clause 2’s job.

Clause 2: `mkSecret` succeeds exactly on in-range input

```
∀ (G : DHGroup) (a : Nat), (mkSecret G a).isSome = true ↔ (1 ≤ a ∧ a < G.q)
```

Syntax. Type-check the left side step by step. `mkSecret G a` has type `Option (Secret G)`; `.isSome` turns it into a `Bool` (Section 4); `= true` turns that into a proposition. The right side is a proposition about `a`. The $\leftrightarrow$ relates the two propositions, so the clause compares *whether the call succeeds* with *whether the number is in range*, and says nothing about the value returned on success.

Intuition. `mkSecret` accepts exactly the numbers in $[1, q - 1]$ and rejects everything else.

Connections. The two directions do different amounts of work.

- *Right to left* (in range implies success) is the genuinely new content. It rules out the always-`none` function that clause 1 alone permits: a `mkSecret` that never succeeds is safe but useless.
- *Left to right* (success implies in range) is already implied by the rest of the specification. If `mkSecret G a = some s`, then `s.inRange` proves $1 \leq s.a < q$, and clause 1 gives `s.a = a`; so `a` is in range. The type `Secret G` enforces this half on its own.

Clause 3: `publicOf` computes $g^a \bmod p$

```
∀ (G : DHGroup) (s : Secret G), (publicOf G s).y = powModRef G.g s.a G.p
```

Syntax. For every group and every secret for it: `publicOf G s` has type `PublicValue G`, `.y` reads its number, and the right side is $g^a \bmod p$ written with the reference function. The equation is between natural numbers.

Intuition. The number a party transmits is exactly $g^a \bmod p$, the textbook Diffie–Hellman public value.

Connections. The *validity* of the output is not this clause’s job: the return type `PublicValue G` already guarantees it, and the implementation had to prove it ([Obligations that live in types](#)). What the clause adds is *which* valid value. It is also the clause the earlier version of the project lacked: there, nothing constrained the transmitted value on its own. It is, finally, derivable from clause 6 and `KeyCorrect`, as shown at the end of this section; it is stated anyway so that the meaning of the value on the wire is explicit.

Clause 4: `parsePublic` returns the number it was given

```
∀ (G : DHGroup) (y : Nat) (P : PublicValue G), parsePublic G y = some P → P.y = y
```

Syntax. The exact counterpart of clause 1: an equation between two values of type `Option (PublicValue G)` as hypothesis, an equation between natural numbers as conclusion.

Intuition. If parsing the received number `y` succeeds, the validated value contains `y` itself.

Connections. Without this clause, the following would satisfy every other clause, yet replace every peer value by `g`:

```
parsePublic G y := if ValidPublic G y then some (G.g, _) else none
```

Each party's key would then be $g^a \bmod p$, its *own* public value, again in the clear. Like clause 1, this clause is satisfied vacuously by a function that always fails.

Clause 5: parsePublic succeeds exactly on valid input

```
∀ (G : DHGroup) (y : Nat), (parsePublic G y).isSome = true ↔ ValidPublic G y
```

Syntax. The same shape as clause 2: a Bool fact made into a proposition, related by $\leftrightarrow$ to the proposition ValidPublic G y.

Intuition. parsePublic accepts a received number exactly when it passes full validation: between 1 and $p - 1$, and in the order- q subgroup (Section 7).

Connections. Again the directions differ.

- *Right to left* (valid implies accepted) is the new content: it rules out a parsePublic that rejects honest peers, which would be safe but would make every exchange fail.
- *Left to right* (accepted implies valid) follows from the type plus clause 4: an accepted P carries the proof P.valid of ValidPublic G P.y, and P.y = y.

So the safety half of validation, *never accept an invalid value*, is guaranteed by the type PublicValue alone. The specification's clauses add completeness.

Clause 6: shared computes $y^a \bmod p$

```
∀ (G : DHGroup) (s : Secret G) (P : PublicValue G), shared G s P = powModRef P.y s.a G.p
```

Syntax. For every group, every secret, and every validated peer value: an equation between natural numbers, with the right side $y^a \bmod p$ written with the reference function.

Intuition. The key is the peer's validated value raised to one's own secret, modulo p .

Connections. The clause covers *every* PublicValue, not only those produced by publicOf, so shared cannot treat values arriving from the network differently from locally produced ones. Mathematically this coverage would follow anyway, since every acceptable value is a power of g (Section 7), but the specification states it rather than leaving the reader to derive it. Combined with clause 3 it yields KeyCorrect, below.

What the six clauses add up to

They determine the four functions completely. Suppose two implementations both satisfy SchemeCorrect. Then:

- mkSecret: on in-range a , clause 2 says it returns *some* secret and clause 1 says that secret's number is a ; by proof irrelevance (Section 5) there is only one such secret. On other inputs, clause 2 forces none.
- parsePublic: the same argument with clauses 4 and 5.
- publicOf: clause 3 fixes the number, and the proof part is irrelevant.
- shared: clause 6 fixes it outright.

So the two implementations agree on every input, and by function extensionality are equal: **the specification has exactly one model**. This is the precise sense in which an implementation can be trusted without being read. Implementations may still differ in *how* they compute (speed, memory, timing), but never in *what* they compute. Those “how” properties are exactly the ones this specification cannot express (Part V).

Clauses 1 and 4 protect the actual program. KeyCorrect survives dropping either of them, because it never mentions mkSecret or parsePublic. But Main.lean obtains secrets only through mkSecret and peer values only through parsePublic, so it is clauses 1 and 4 that keep its keys secret.

There is deliberate redundancy. Besides the left-to-right halves of clauses 2 and 5, already implied by the types, clause 3 follows from clause 6 and KeyCorrect: take Alice’s secret to be 1; then KeyCorrect says Bob’s public value y satisfies $y \bmod p = g^b \bmod p$, and since a valid y is below p , this forces $y = g^b \bmod p$. Redundancy is harmless and chosen on purpose: each redundant statement makes something explicit that a reader would otherwise have to derive. Finding such dependencies is itself a good test of having understood a specification.

What is not there. No clause says where secrets come from. It cannot, since “uniformly random” is a statement about distributions and needs a probabilistic framework; Part IV shows where that obligation lands instead.

KeyCorrect

```
def KeyCorrect
  (publicOf : (G : DHGroup) → Secret G → PublicValue G)
  (shared   : (G : DHGroup) → Secret G → PublicValue G → Nat) : Prop :=
  ∀ (G : DHGroup) (sA sB : Secret G),
    shared G sA (publicOf G sB) = powModRef G.g (sA.a * sB.a) G.p
```

Syntax. Higher-order over two of the four functions, quantifying over a group and two secrets for it.

Intuition. The headline guarantee: Alice, combining her secret with Bob’s public value, obtains $g^{ab} \bmod p$.

Connections. It is a *consequence* of clauses 3 and 6 of SchemeCorrect: Bob’s public value is $g^b \bmod p$, and Alice’s key is then $(g^b \bmod p)^a \bmod p = g^{ab} \bmod p$ by the commutative square of Section 6. It is included because it is the statement a reader believes DH to be, and the converse direction was noted above. Agreement between the parties follows as well: since $ab = ba$ in $\mathbb{N}$, the key Bob computes is the same. The earlier version of this project stated agreement as a separate conjunct and noted it was nearly vacuous by itself, satisfied by a scheme whose “key” is always 0. Pinning the key to g^{ab} is what gives the guarantee content.

Obligations that live in types, not in theorem statements

Reading the specification is not finished when its definitions have been read: the *stub* fixes the types of the functions the agent must write, and several of those types carry proof obligations that no theorem mentions.

- **defaultGroup : DHGroup** obliges the agent to produce a certified-good group: primality proofs for a 127-bit and a 128-bit number, and the order condition on g . The specification never

mentions this as a theorem.

- **publicOf** : $\dots \rightarrow \text{PublicValue } G$ obliges a proof that $g^a \bmod p$ is *always* valid: not 1, not $p - 1$, and in the subgroup, for every secret. The argument is nontrivial: g has order exactly q , so $g^a \neq 1$ for $1 \leq a < q$, and $g^a \neq p - 1$ because $p - 1$ has order 2.
- **parsePublic**, **mkSecret** oblige proofs that the values they construct satisfy their invariants.

A function whose output is a proof-carrying structure is a function that cannot be written without a proof. This is the mechanism, and the reason the shell cannot cheat; it also means a specification's real content is spread between its propositions and the types of the functions it constrains.

The three theorems, and the locked section

The stub requires `powMod_correct_impl`, `scheme_correct_impl`, and `key_correct_impl`, each applying a higher-order specification to the stub's own functions: turning "a property any function might have" into "a claim about *these* functions". The locked section restates them under shorter names, as in [Section 10](#).

What the prompt asks that the specification cannot express

- **Square-and-multiply, reduced at every step; kernel-evaluable definition.** An efficiency and proof-engineering requirement. Checked by running at 127-bit sizes, where anything naive would never finish.
- **powModRef only in statements and proofs.** Checked by grep on `Main.lean`, and indirectly by running.
- **No sorry, admit, native_decide, new axioms, implemented_by, extern, unsafe, dbgTrace.** Each would enlarge what you must trust, and each is scanned for mechanically ([Part V](#)). `native_decide` in particular would put Lean's compiler into the trusted base and show up as the axiom `Lean.ofReduceBool`.
- **Entropy from the operating system.** Not checkable by any theorem; reviewed in `Main.lean` ([Part IV](#)).

Part IV — The unverified shell: `Main.lean`

```
import DHSafe.Impl

/-- Big-endian conversion of a byte array to a natural number. -/
def bytesToNat (b : ByteArray) : Nat :=
  b.foldl (fun acc x => acc * 256 + x.toNat) 0

/-- Draw a fresh secret from 32 bytes of OS randomness, mapped into `[1, q - 1]`. -/
def freshSecret (G : DHGroup) : IO (Option (Secret G)) := do
  let bytes ← IO.getRandomBytes 32
  let a := bytesToNat bytes % (G.q - 1) + 1
  return Impl.mkSecret G a

def main : IO UInt32 := do
```

```

let G := Impl.defaultGroup
IO.println s!"p = {G.p}"
IO.println s!"q = {G.q}"
IO.println s!"g = {G.g}"

let some sA ← freshSecret G
  | IO.eprintln "error: failed to create Alice's secret"; return 1
let some sB ← freshSecret G
  | IO.eprintln "error: failed to create Bob's secret"; return 1
IO.println s!"Alice secret (demo only) = {sA.a}"
IO.println s!"Bob secret (demo only) = {sB.a}"

let pubA := Impl.publicOf G sA
let pubB := Impl.publicOf G sB
IO.println s!"Alice public = {pubA.y}"
IO.println s!"Bob public = {pubB.y}"

-- Transmission: only the plain numbers cross the wire; each receiver re-validates.
let wireA : Nat := pubA.y
let wireB : Nat := pubB.y
let some bobFromAlice := Impl.parsePublic G wireA
  | IO.eprintln "error: Bob rejected Alice's public value"; return 1
let some aliceFromBob := Impl.parsePublic G wireB
  | IO.eprintln "error: Alice rejected Bob's public value"; return 1

let keyA := Impl.shared G sA aliceFromBob
let keyB := Impl.shared G sB bobFromAlice
IO.println s!"Alice key = {keyA}"
IO.println s!"Bob key = {keyB}"
IO.println s!"keys equal: {keyA == keyB}"

IO.println "parsePublic rejection table:"
for y in [0, 1, G.p - 1, G.p, G.p + 1, 2, 3, G.g] do
  let verdict := if (Impl.parsePublic G y).isSome then "accepted" else "rejected"
  IO.println s!" y = {y}: {verdict}"
return 0

```

bytesToNat

Syntax. A pure function $\text{ByteArray} \rightarrow \text{Nat}$. `b.foldl f 0` folds the function `f` over the bytes from left to right starting from the accumulator `0`; `fun acc x => acc * 256 + x.toNat` is an anonymous function (a lambda) of the running total and the next byte, and `x.toNat` converts a byte to a natural number.

Intuition. Reads the bytes as a big-endian base-256 numeral: the standard way to turn raw randomness into an integer.

Connections. Entirely ordinary code with no proof behind it. It matters only through its use in

freshSecret.

freshSecret

Syntax. Of the dependent type $(G : \text{DHGroup}) \rightarrow \text{IO}(\text{Option}(\text{Secret } G))$ (Section 1): an effectful computation producing an *optional* secret for the given group. `IO.getRandomBytes 32` is the action that asks the operating system for 32 random bytes; `let bytes ← ...` runs it (Section 11). `return Impl.mkSecret G a` wraps the verified function’s optional result as the action’s value.

Intuition. Draw 256 bits of OS entropy, reduce them into $[1, q - 1]$ by $n \bmod (q - 1) + 1$, and let the verified `mkSecret` produce the `Secret`. The reduction maps $[0, q - 2]$ to $[1, q - 1]$, the exact range the type requires. Because 256 bits far exceed the 127 bits of q , the bias introduced by the reduction is below about 2^{-129} . The truncated subtraction $G.q - 1$ is harmless since $q > 2^{126}$.

Connections. This is where the one obligation that *no type can carry* is discharged: choosing the secret unpredictably. The specification’s `Secret` records only the range. That the OS is a good entropy source is an assumption, and nothing in this project verifies it. The function cannot be used to bypass anything: it ends in `Impl.mkSecret`, whose result the type system forces the caller to handle as a possible none.

main

Syntax. Of type `IO UInt32`, returning a process exit code. The line `let some sA ← freshSecret G | ... ; return 1` is the pattern-bind of Section 11: if the action yields none, print an error and exit with code 1. The lines `let some bobFromAlice := Impl.parsePublic G wireA | ...` use the pure variant, since `parsePublic` is an ordinary function rather than an action. `Impl.publicOf G sA` is a pure call returning a `PublicValue G`; `.y` reads its number. The final `for` loop prints one line per test value.

Intuition. One complete exchange, Section 6’s diagram executed literally: each party draws a secret and computes a public value; the *plain numbers* cross a simulated wire; each receiver re-validates with `parsePublic` before doing anything else; each then combines the peer’s validated value with its own secret; the keys are printed and compared. A final table feeds eight chosen numbers to `parsePublic` to show what it accepts and rejects.

Connections. This is the “unverified shell” box of Section 12, made concrete. What the types now guarantee, so that review need not cover it:

- *Validation cannot be skipped.* `Impl.shared` takes a `PublicValue`, and the only way to obtain one from a received number, short of writing a validity proof by hand (which the shell does not do; Part V checks this by `grep`), is `Impl.parsePublic`. It returns none on invalid input, and the code is forced to handle that case.
- *Arguments cannot be passed in the wrong positions.* `Secret G` and `PublicValue G` are different types.
- *Parameters cannot be unvetted.* Every call takes a `DHGroup`.

What review must still establish, because nothing covers it:

- **Every value comes through `Impl`.** It does: group, secrets, public values, parsed values, and keys all come from `Impl.*`, and a `grep` confirms no direct constructors and no mention of `powModRef`.

- **The pairing of secret and peer value is right.** Alice’s key combines her secret `sA` with `aliceFromBob`, Bob’s parsed value; Bob’s combines `sB` with `bobFromAlice`. Both have the same type, so a program that used a party’s *own* public value in place of the peer’s would compile. It does not.
- **Failure paths exit nonzero.** All four error branches return 1.

Two optional improvements, neither affecting correctness. The program prints `keys equal: false` but would still exit with code 0 if the keys ever differed, which the theorem makes impossible, though a script checking the exit status would prefer a nonzero code. And the rejection table, as written, never exercises the *subgroup* test: the five rejected values fail the range test, and 2, 3, 4 all happen to lie in the subgroup. [Part V](#) does the missing test directly.

Part V — Verification, as actually performed

The protocol, in order

1. Create the Mathlib project from the template; build once.
2. Write `Spec.lean` and the `Impl.lean` stub by hand; build; read the specification critically ([Part III](#)).
3. Turn off Mathlib’s style linters in the lakefile (so “zero warnings” is satisfiable), and give the project its own git repository.
4. Commit; make the specification and build configuration immutable.
5. Run the agent.
6. Verify independently, as below; never rely on the agent’s own report.

Step 1: the agent stayed inside its permissions

```
$ git --no-pager status --short
M DHSafe/Impl.lean
M Main.lean
$ git --no-pager log --oneline
ecd663e (HEAD -> master) disable Mathlib style linters
c8a187a spec + stubs
$ lsattr DHSafe/Spec.lean lakefile.toml lean-toolchain lake-manifest.json
----i-----e----- DHSafe/Spec.lean
----i-----e----- lakefile.toml
----i-----e----- lean-toolchain
----i-----e----- lake-manifest.json
```

Exactly two modified files, both history commits intact, all four locked files still immutable.

Step 2: mechanical scans instead of reading

The commands below all printed nothing. (The denylist was run as a single grep; it is shown here as two, each wrapped, to fit the page.)

```

grep -nE "sorry|admit|native_decide|axiom|implemented_by|extern|unsafe" \
  DHSafe/Impl.lean Main.lean
grep -nE "dbgTrace|dbg_trace|opaque|initialize|csimp|macro|elab|syntax|open Lean" \
  DHSafe/Impl.lean Main.lean
grep -nE "DHGroup\.mk|Secret\.mk|PublicValue\.mk|powModRef" Main.lean
grep -n "set_option" DHSafe/Impl.lean Main.lean
lake env lean DHSafe/Impl.lean
lake env lean Main.lean

```

In order: no forbidden construct in either file; no direct construction of the proof-carrying types and no mention of `powModRef` in the shell; no compiler option changed, so no heartbeat or recursion-depth workaround was needed; and both files compile, invoked directly, with no warnings or errors.

Step 3: the axiom audit, including the executable

```

cat > AxCheck.lean << 'EOF'
import DHSafe.Impl
#print axioms Impl.powMod_correct_impl
#print axioms Impl.scheme_correct_impl
#print axioms Impl.key_correct_impl
#print axioms Impl.defaultGroup
EOF
lake env lean AxCheck.lean

```

and similarly import `Main` with `#print axioms main`. The output:

```

'Impl.powMod_correct_impl' depends on axioms: [propext, Classical.choice, Quot.sound]
'Impl.scheme_correct_impl' depends on axioms: [propext, Classical.choice, Quot.sound]
'Impl.key_correct_impl' depends on axioms: [propext, Classical.choice, Quot.sound]
'Impl.defaultGroup' depends on axioms: [propext, Classical.choice, Quot.sound]
'main' depends on axioms: [propext, Classical.choice, Quot.sound]

```

Only Lean's three standard axioms. `Impl.defaultGroup` is included deliberately: its primality proofs are exactly where a `native_decide` shortcut (`axiom Lean.ofReduceBool`) would appear, and `main` is included because the executable, too, must rest on nothing else. `Classical.choice` appears here, unlike in the earlier version of the project, because the number-theory proofs use `Mathlib`, which uses classical reasoning freely.

Step 4: the program, run twice

```

p = 207386738379149228107986191036893352879
q = 103693369189574614053993095518446676439
g = 4
Alice secret (demo only) = 2358326084105449998015536719583807939
Bob secret   (demo only) = 55220411651017949108578776414515153666
Alice public = 151442588175696060037101206875783240210
Bob public   = 125750500349277838292834373088768164367

```

```

Alice key = 20224145139083631102501839896278838389
Bob key   = 20224145139083631102501839896278838389
keys equal: true
parsePublic rejection table:
  y = 0: rejected
  y = 1: rejected
  y = 207386738379149228107986191036893352878: rejected
  y = 207386738379149228107986191036893352879: rejected
  y = 207386738379149228107986191036893352880: rejected
  y = 2: accepted
  y = 3: accepted
  y = 4: accepted

```

A second run produced different secrets and public values, and again equal keys. Completing at all, with exponents of about 127 bits, confirms the efficiency requirement of [Section 9](#): anything that formed g^e in full would never have finished.

Step 5: an independent cross-check

The following Python script shares no code with Lean. It re-derives the group's properties and every printed number for both runs, and the program's rejection table, from the specification's own definitions:

```

# Independent cross-check of the DHSafe run outputs. Shares no code with Lean.
p = 207386738379149228107986191036893352879
q = 103693369189574614053993095518446676439
g = 4

def miller_rabin(n): # strong probable-prime test with the first 20 prime bases
    bases = (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71)
    for b in bases:
        if n % b == 0:
            return n == b
    d, r = n - 1, 0
    while d % 2 == 0:
        d //= 2
        r += 1
    for b in bases:
        x = pow(b, d, n)
        if x in (1, n - 1):
            continue
        for _ in range(r - 1):
            x = x * x % n
            if x == n - 1:
                break
        else:
            return False
    return True

```

```

def valid(y): # the specification's ValidPublic
    return 1 < y and y + 1 < p and pow(y, q, p) == 1

# the group satisfies GoodGroup
assert miller_rabin(p) and miller_rabin(q) and p == 2 * q + 1
assert 2**126 < q and 1 < g < p and pow(g, q, p) == 1

# the two runs, as printed by the program: (a, b, A, B, key)
runs = [
    (2358326084105449998015536719583807939, 55220411651017949108578776414515153666,
     151442588175696060037101206875783240210, 125750500349277838292834373088768164367,
     20224145139083631102501839896278838389),
    (82760244147453805052306895629361056714, 55572118330761423776625885672573129947,
     2591376621224744350179320639308871424, 76232178768902566793359700889942339666,
     21316608112366615136327069094136066445),
]
for a, b, A, B, key in runs:
    assert 1 <= a < q and 1 <= b < q
    assert pow(g, a, p) == A and pow(g, b, p) == B
    assert pow(B, a, p) == key == pow(A, b, p) == pow(g, a * b, p)
    assert valid(A) and valid(B)

# the program's rejection table
table = {0: False, 1: False, p - 1: False, p: False, p + 1: False,
         2: True, 3: True, g: True}
assert all(valid(y) == ok for y, ok in table.items())

# values that fail only the subgroup test, for the extra check
print("smallest non-subgroup values:", [y for y in range(2, 21) if not valid(y)])
print("all checks passed")

```

Running it prints:

```

smallest non-subgroup values: [11, 17, 19]
all checks passed

```

The group satisfies GoodGroup independently of Lean (with probabilistic primality testing, which is evidence rather than proof, but here only a cross-check of what Lean proved); both secrets lie in $[1, q - 1]$; both public values equal $4^{\text{secret}} \bmod p$; both parties' keys equal $4^{ab} \bmod p$; and the rejection table matches, entry by entry.

Step 6: closing the gap in the demonstration

Because the table never exercises the subgroup test, the smallest values failing $y^q \equiv 1$ were computed independently (11, 17, 19) and fed to `parsePublic` directly, together with three values known to pass:

```

$ cat Check.lean
import DHSafe.Impl
#eval [11, 17, 19, 2, 3, 4].map fun y => (y, (Impl.parsePublic Impl.defaultGroup y).isSome)

$ lake env lean Check.lean
[(11, false), (17, false), (19, false), (2, true), (3, true), (4, true)]

```

The subgroup branch of validation is now exercised in both directions, by a run and, separately, by the theorem.

Workflow pitfalls encountered

These cost time and are worth recording.

- **A project inside another repository.** `lake new` did not create a git repository for DHSafe, because it sat inside the home directory, which was itself a git repository. Every git command therefore operated on the *home* directory: `git status` listed shell configuration files, and `git add -A` began walking caches and toolchains, which hung the terminal. The fix is `git init` inside the project, and the habit is to check `git rev-parse --show-toplevel` before the first commit. Nothing was committed to the wrong repository, but only by luck.
- **Style linters versus “zero warnings”.** The Mathlib template enables style linters that complain about formatting in files the agent may not edit. They were disabled in the lakefile *before* the build configuration was locked.
- **Lock the build configuration, not only the specification.** The lakefile, toolchain file, and manifest were made immutable too, so the agent could not change compiler options; the prompt told it where it may set options instead.
- **lake exe re-checks dependencies on every run.** Each `lake exe dhsafe` re-examined thousands of Mathlib build jobs before running the program. Running the compiled binary directly, `.lake/build/bin/dhsafe`, avoids this.
- **Interrupting with Ctrl+C, not Ctrl+Z.** Ctrl+Z suspends a process and leaves it lingering; it does not stop it.

What has been established, and what has not

Established, by theorem. `powMod` equals $g^e \bmod p$ for every input with $p > 0$. `mkSecret` and `parsePublic` succeed exactly on in-range and valid input respectively, and return the number they were given. `publicOf` returns $g^a \bmod p$ and `shared` returns $y^a \bmod p$ for every validated peer value; hence the key equals $g^{ab} \bmod p$. The group the implementation supplies satisfies `GoodGroup`: both primes are proven prime, $p = 2q + 1$, $q > 2^{126}$, and g has the right order.

Established by the type system, without review. Every `PublicValue` is validated and every `Secret` is in range. No key can be computed from an unvalidated number, no operation can be applied to unvetted parameters, and a secret and a peer value cannot be passed in the wrong argument positions.

Established mechanically. The specification and build configuration were untouched; the denylist scans are empty; only the three standard axioms appear, for the theorems, the group, and the executable; an independent implementation agrees on every printed value; the subgroup test was exercised in both directions.

Not established by any theorem. All but the pairing item below are not even expressible in this specification.

- *Secrecy of the key.* No hardness assumption appears anywhere.
- *Entropy quality.* The shell reads the operating system’s source; nothing verifies that source, or that the secrets are uniformly distributed.
- *Side channels.* Arbitrary-precision arithmetic is not constant-time, and a functional specification cannot even express timing.
- *The demonstration prints secrets*, labelled “demo only”.
- *Suitability of the parameters.* The group is a toy (127-bit q , 128-bit p). The size floor in the specification must be raised to realistic values before the guarantee means anything against an adversary.
- *The pairing of secret with peer value in Main.lean:* correct, but confirmed by review only.
- *Authentication.* Unauthenticated Diffie–Hellman is open to a man in the middle however correct the arithmetic.

Natural next steps

- **Raise the floor.** Changing `qFloor` is a one-line edit to the specification, and for safe-prime groups the right value is 2^{2046} , not the 2^{223} its own comment suggests (see `qFloor` in [Part III](#)). The implementation’s burden grows: a primality proof for a 2048-bit safe prime needs certificates of a very different scale. This is the real cost of the quantitative guarantee, and the toy group exists so the *shape* of the proof could be exercised first.
- **Negotiate among several certified groups**, replacing the single `defaultGroup` by a list the specification constrains.
- **A genuine security proof**, such as ElGamal’s IND-CPA security from the decisional Diffie–Hellman assumption, in a probabilistic framework (EasyCrypt, CryptHOL, SSProve, or research libraries in Lean).
- **Authenticated key exchange**, with a symbolic protocol verifier or a specification of the authentication layer.

References

1. W. Diffie and M. E. Hellman, “New Directions in Cryptography”, *IEEE Transactions on Information Theory* 22(6), 1976, 644–654. [doi:10.1109/TIT.1976.1055638](https://doi.org/10.1109/TIT.1976.1055638). The original key-agreement scheme of [Section 6](#).
2. S. Pohlig and M. Hellman, “An Improved Algorithm for Computing Logarithms over $GF(p)$ and Its Cryptographic Significance”, *IEEE Transactions on Information Theory* 24(1), 1978. [doi:10.1109/TIT.1978.1055817](https://doi.org/10.1109/TIT.1978.1055817). Why the largest prime factor of the group order is what matters ([Section 7](#)).
3. J. M. Pollard, “Monte Carlo Methods for Index Computation (mod p)”, *Mathematics of Computation* 32(143), 1978, 918–924. [doi:10.1090/S0025-5718-1978-0491431-9](https://doi.org/10.1090/S0025-5718-1978-0491431-9). The rho method and its square-root cost ([Section 7](#)).
4. V. R. Pratt, “Every Prime Has a Succinct Certificate”, *SIAM Journal on Computing* 4(3), 1975. [doi:10.1137/0204018](https://doi.org/10.1137/0204018). Lucas-style certificates applied recursively ([Section 8](#)).
5. NIST SP 800-56A Rev. 3, *Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete*

- Logarithm Cryptography*, 2018. [doi:10.6028/NIST.SP.800-56Ar3](https://doi.org/10.6028/NIST.SP.800-56Ar3). Public-value validation and approved safe-prime groups ([Section 7](#), `ValidPublic`).
6. NIST SP 800-57 Part 1 Rev. 5, *Recommendation for Key Management: Part 1 – General*, 2020. [doi:10.6028/NIST.SP.800-57pt1r5](https://doi.org/10.6028/NIST.SP.800-57pt1r5). Security strengths of key sizes, the source of “2048-bit / 224-bit” (`qFloor`).
 7. D. K. Gillmor, RFC 7919, *Negotiated Finite Field Diffie-Hellman Ephemeral Parameters for Transport Layer Security (TLS)*, 2016. rfc-editor.org/rfc/rfc7919. Standard 2048-bit and larger safe-prime groups.
 8. The Mathlib Community, *Mathlib documentation*. leanprover-community.github.io/mathlib4_docs. Source of `Nat.Prime` and the number theory used by the implementation.